

autonomy-light

파라미터 및 사용 가이드

Livox MID360/MID360s + Point-LIO 기반 로봇 중심 Height Map 런타임

기준: 현재 저장소의 `config/autonomy_light.yaml` · 작성일: 2026-07-26 · 단위: 별도 표기가 없으면 m / s / rad

기준값과 적용 위치

표의 기본값은 현재 YAML 에서 읽었다. 설치된 Jetson 에서는 `/etc/cocelo/autonomy-light/autonomy_light.yaml` 을 수정한다. 변경값은 시작 시에만 읽으므로 프로세스를 재시작해야 한다.

1. 시스템 개요와 빠른 시작

LiDAR → Livox driver → Point-LIO → 등록 scan / 전역 정밀 지도 → height map 생성 → 외부 ROS 2 제어 출력

운용 전 확인

- LiDAR NIC 이름과 IP 를 확인하고 `livox_interface`, `livox_host_ip`, `livox_lidar_ip` 을 맞춘다.
- LiDAR 장착 자세와 `target_to_lidar_xyz/rpy` 를 실제 `base_link` 기준으로 맞춘다.
- 수신 제어기의 `ROS_DOMAIN_ID` 를 `external_ros_domain_id` 와 일치시킨다.

```
source /opt/ros/humble/setup.bash

source ~/ros2_ws/install/setup.bash

./launch.sh --real          # YAML 의 LiDAR 모델 사용

./launch.sh --real --mid360s --vis # 모델 override + HeightMap 뷰어

# Debian 설치 환경

autonomy-light --real

autonomy-light --real --map "$HOME/autonomy-light-maps/site.pcd"
```

상황

명령 / 옵션

설명

실제 로봇	<code>--real</code>	기본 모드. Livox driver 와 Point-LIO 를 시작한다.
시뮬레이션	<code>--sim --sim-topic-prefix /f4</code>	driver 를 시작하지 않고 시뮬레이터 LiDAR/IMU topic 을 쓴다. root topic 이면 prefix 를 ""으로 지정한다.
노드만 점검	<code>--no-drivers</code>	Livox 및 Point-LIO 없이 autonomy_light 만 실행한다.
설정 파일 교체	<code>--config FILE</code>	실험용 YAML 을 지정한다.
저장 지도	<code>--map FILE.pcd</code>	초기 재지역화 성공 전까지 height map 출력은 보류된다.

2. 설정 변경, 검증, 지도 생성

```

sudo nano /etc/cocelo/autonomy-light/autonomy_light.yaml

# 실행 터미널에서 Ctrl+C 후

autonomy-light --real

ROS_DOMAIN_ID=0 ros2 topic hz /autonomy_light/height_map_data

ROS_DOMAIN_ID=0 ros2 topic echo /autonomy_light/heartbeat --once

```

튜닝 원칙

한 번에 한 계열만 변경하고, 값·주행 환경·결과를 기록한다. 좌표계/장착값이 틀린 상태에서는 필터 파라미터를 바꿔도 해결되지 않는다.

저장 지도 만들기와 사용

```

mkdir -p "$HOME/autonomy-light-maps"

autonomy-light-mapping --real --output "$HOME/autonomy-light-maps/site.pcd"

# 경로 주행 → Ctrl+C 한 번 → pose-graph 완료 로그까지 대기

autonomy-light --real --map "$HOME/autonomy-light-maps/site.pcd"

```

site.pcd 는 refined 지도, site.raw.pcd 는 복구/진단용 원본이다. heartbeat 가 ready:...가 될 때까지 저장 지도 기반 제어 입력을 사용하지 않는다.

3. 좌표계, 지도 형상, 출력 메시지

파라미터	기본값	용도 및 설정 기준
target_frame	base_link	제어 기준 로봇 frame. 수신 제어기의 기준과 같아야 한다.
height_map_frame	base_link_gravity	roll/pitch 를 제거한 중력 정렬 frame 으로 바닥 흔들림을 줄인다.
lidar_frame / lidar2_frame	lidar_link / lidar2_link	각 LiDAR frame 이름. URDF/TF 와 일치해야 한다.
target_to_lidar_xyz	[0.038335, 0.00162081, 0.138986]	base_link 에서 LiDAR 1 까지 변환. z 오차는 바닥/장애물 높이를 함께 왜곡한다.
target_to_lidar_rpy	[3.1416, 0, 0]	LiDAR 1 장착 roll,pitch,yaw. 뒤집힌 장착이면 roll 보정이 핵심이다.
target_to_lidar2_xyz/rpy	LiDAR 1 과 동일	두 번째 LiDAR 에만 사용한다.
elevation_resolution	0.01	cell 크기. 작을수록 edge 는 선명하나 point 부족·계산량·구멍이 증가한다.
elevation_x_length/ y_length	1.6 / 0.8	로봇 중심 grid 크기. shape 는 160 × 80, 총 12,800 cells 다.
publish_rate_hz	50.0	외부 height map 발행률. 수신 처리 능력이 부족하면 낮춘다.

HeightMap 해석

외부 제어 메시지는 /autonomy_light/height_map_data, 타입은 autonomy_light/msg/HeightMap 이다. 필드는 header, float32[] data, resolution, x_length, y_length 다.

```
width = ceil(x_length / resolution)

height = ceil(y_length / resolution)

index = row * width + col    # row: y_min → y_max, col: x_min → x_max

x = -0.5*x_length + (col+0.5)*resolution

y = -0.5*y_length + (row+0.5)*resolution

data[index] = clipping.max_z - cell_z
```

현재 clipping.max_z=0.48 이므로 평지는 대체로 0.48 에 가깝고, 높은 장애물일수록 0.0 쪽으로 작아진다. data 는 raw z 가 아니라 거리형 값이다.

4. ROS 2 domain, topic, LiDAR 연결

파라미터	기본값	설명
<code>internal_ros_domain_id</code>	0	LiDAR 및 Point-LIO 내부 domain.
<code>external_ros_domain_id</code>	0	제어 출력 domain. 분리가 필요하면 예: internal 42, external 0 으로 설정한다.
<code>point_lio_global_map.ros_domain_id</code>	0	전역 지도 RViz/debug domain. 제어 domain 과 분리하면 큰 PointCloud 유입을 피한다.
<code>livox_model</code>	<code>mid360</code>	<code>mid360</code> 또는 <code>mid360s</code> . 실행 옵션이 YAML 보다 우선한다.
<code>livox_interface</code>	<code>enP8p1s0</code>	LiDAR 1 유선 NIC 이름.
<code>livox_host_ip / livox_lidar_ip</code>	<code>192.168.1.50/24 / 192.168.1.166</code>	Jetson LiDAR 망 IP/CIDR 와 LiDAR 1 IP.
<code>livox2_interface / host2_ip / lidar2_ip</code>	빈 문자열	두 번째 LiDAR 용. lidar2 IP 가 비어 있으면 단일 LiDAR 로 동작한다.
<code>raw_lidar_topic / raw_imu_topic</code>	<code>livox/lidar / livox/imu</code>	LiDAR 1 raw 입력.
<code>raw_lidar2_topic</code>	빈 문자열	비어 있지 않으면 cloud merge 가 활성화된다.
<code>lidar_merge.sync_tolerance / max_queue_size</code>	<code>0.005 / 8</code>	두 cloud timestamp 허용차(s)와 동기화 queue. sync miss 가 잦으면 tolerance 를 조금 올리되 움직임 왜곡에 주의한다.

외부로 나가는 주요 topic: `/autonomy_light/height_map_data`, `/height_map`, `/odom`, `/path`, `/tf`, `/tf_static`. HeightMap/PointCloud2 는 Reliable depth 2, odom/path 는 Reliable depth 10 이다.

5. 높이 기준과 Height Map 알고리즘

5.1 Height origin

파라미터	기본값	튜닝 효과
<code>height_origin.mode</code>	<code>local_floor</code>	주변 바닥으로 z 기준 추정. odom z jitter 에 강하다. odom 은 Point-LIO z, <code>fixed</code> 는 <code>fixed_z</code> 사용.
<code>fixed_z</code>	0.0	mode=fixed 일 때만 사용.

<code>filter_alpha / max_step</code>	<code>0.18 / 0.015</code>	클수록 빨리 따라가나 흔들림도 증가. <code>max_step</code> 은 frame 당 z 기준 최대 변화다.
<code>floor_radius / percentile / min_points</code>	<code>0.6 / 0.20 / 20</code>	바닥 추정 범위·낮은 값 선택 정도·최소 point. 부족하면 odom z fallback 이 가능하다.

5.2 Cell 선택 및 clipping

파라미터	기본값	튜닝 효과
<code>algorithm.elevation_backend</code>	<code>autonomy_min_z</code>	낮은 지지 표면을 terrain 으로 선택하고 장애물은 별도로 overlay 한다.
<code>clipping.enabled / min_z / max_z</code>	<code>true / 0.0 / 0.48</code>	최종 출력 범위. <code>max_z</code> 는 HeightMap 거리형 값의 기준 높이이기도 하다.
<code>min_points_per_cell</code>	<code>3</code>	terrain 최소 지지점. 올리면 노이즈는 감소하고 빈 cell 은 증가한다.
<code>supported_min_enabled / support_band</code>	<code>true / 0.025</code>	낮은 반환점이 이웃 낮은 반환점과 2.5 cm 이내에서 지지되는지 확인한다.
<code>obstacle_override_enabled</code>	<code>true</code>	terrain filtering 뒤에도 장애물 top 을 보존한다.
<code>obstacle_min_height / min_points / support_band</code>	<code>0.035 / 1 / 0.06</code>	낮출수록 작은 장애물에 민감해지고 false positive 가능성이 증가한다.
<code>obstacle_projection_radius_cells</code>	<code>0</code>	장애물을 이웃 cell 로 확장할 반경. 0 은 edge 번짐을 줄인다.

6. 보간, 노이즈 제거, 평활화

계열 / 파라미터	기본값	설명 및 권장 조정
<code>interpolation_max_passes / min_neighbors / max_height_diff</code>	<code>4 / 3 / 0.02</code>	짧고 평탄한 빈틈 보간. 장애물 잔상·edge 번짐이 있으면 pass 를 줄이거나 높이차 한계를 낮춘다.
<code>fill_remaining_height</code>	<code>0.0</code>	끝까지 알 수 없는 cell 의 값. 수신 제어기가 이 fallback 을 어떻게 해석하는지 맞춘다.
<code>initial_floor_seed_fill.*</code>	<code>true, 0.20, 0.40, 0.08, 0.70</code>	시작 시 blind cell 을 ROI 옆/밖 ring 의 바닥으로 한 번 seed. 순서: enabled, side_width, search_margin, cluster_band,

		lower_fraction.
frame_aggregation.robust_height_gate / temporal_alpha	0.025 / 0.85	견고한 height gate 와 terrain 시간 평활. alpha 를 낮추면 안정적이나 반응이 늦다.
fill_missing_from_previous_grid	false	이전 grid 로 빈 cell 보완. 동적 장애물 잔상을 피하려면 false 유지.
isolated_filter.radius / min_support_neighbors	2 / 5	5×5 근방 sparse spike 제거. support 수를 올리면 더 깨끗하지만 얇은 edge 가 사라질 수 있다.
isolated_filter.support_height_diff / outlier_height_diff	0.02 / 0.025	이웃과의 높이 호환/이상치 판단 임계값.
hole_fill.radius / min_neighbors / max_height_diff	2 / 5 / 0.02	평탄하고 지지되는 hole 만 메운다.
bilateral.radius / sigma_spatial / sigma_height	2 / 1.3 / 0.012	terrain bilateral smoothing 의 공간/높이 영향 범위.
bilateral.max_height_diff / passes	0.025 / 2	2.5 cm 이상 discontinuity 를 평균내지 않는다. 과도한 smoothing 이면 passes 를 줄인다.

증상별 첫 조치

지지적거림: min_points_per_cell 또는 min_support_neighbors 를 한 단계 올린다. **작은 장애물 반응 지연:**

obstacle_min_height 를 낮춘다. **장애물 잔상:** temporal_alpha 와 interpolation pass 를 줄이고 previous-grid fill 이
false 인지 확인한다.

7. 전역 지도, 재지역화, mapping

계열	기본값	역할
point_lio_global_map.enabled / use_for_height_map	true / true	누적 전역 지도를 만들고 height map source 로 사용한다.
height_voxel_leaf_size / height_max_points	0.01 / 4,000,000	height 추출용 고정밀 저장소. 해상도·메모리의 핵심 trade-off.
voxel_leaf_size / max_points	0.05 / 500,000	일반 전역 지도 RViz copy 의 경량화 값.
refined_visual_voxel_leaf_size / max_points	0.02 / 1,500,000	정밀 지도 시각화 copy. 내부 terrain store 와 별개다.
refinement.mean_k / stddev_multiplier	16 / 0.8	scan 별 isolated return 제거. multiplier 를 작게 하면 더

<code>saved_map_file / saved_map_frame</code>	<code>"" / map</code>	공격적으로 제거한다.
<code>saved_map_localization.max_fitness</code>	<code>0.01</code>	저장 PCD와 좌표 frame. 보통 <code>--map</code> 옵션으로 파일을 지정한다.
<code>initial_submap_duration_sec / min_points</code>	<code>2.0 / 500</code>	GICP 수용 품질 gate. map·extrinsic·현장 geometry를 먼저 확인하고 무작정 완화하지 않는다.
<code>runtime_localization.update_interval_sec / submap_duration_sec</code>	<code>1.0 / 5.0</code>	초기 global matching에 쓸 registered scan 누적량.
<code>runtime_localization.filter_alpha</code>	<code>0.20</code>	저주기 전역 보정 cadence와 rolling submap 길이. Point-LIO odometry는 고주기로 유지된다.
		map→odom 보정 low-pass. 낮추면 부드럽고 느리며 높이면 빠르고 jitter가 증가한다.

Offline mapping (mapping.sh가 주로 override)

파라미터	기본값	설명
<code>mapping_slam.keyframe_distance_m / yaw_deg</code>	<code>0.5 / 7.5</code>	keyframe 생성 거리/회전 기준.
<code>keyframe_voxel_leaf_size / max_points</code>	<code>0.03 / 15000</code>	keyframe cloud 크기 제한.
<code>scan_context.rings / sectors / max_radius</code>	<code>20 / 60 / 40.0</code>	루프 후보 검색 descriptor 해상도/범위.
<code>loop.min_keyframe_separation / query_stride</code>	<code>30 / 5</code>	시간적으로 먼 후보만, 매 5번째 keyframe에서 loop 검사.
<code>loop.max_fitness / min_inlier_fraction</code>	<code>0.025 / 0.35</code>	loop closure 수용 품질 gate.
<code>optimizer.iterations / loop_weight</code>	<code>20 / 20.0</code>	XYZ+yaw pose graph 최적화 반복/loop 제약 가중치.

8. 장애 진단 체크리스트

증상	확인 순서
LiDAR 입력 없음	<code>ip addr</code> 로 NIC/IP 확인 → <code>livox_*</code> 확인 → internal domain에서 raw topic 확인.
HeightMap이 없음	heartbeat → Point-LIO registered/local map → 저장 지도면 relocalization 완료 → <code>height_map_debug.manual_mode</code> 로 수신단만 분리 점검.

지도와 위치가 어긋남	LiDAR extrinsic 과 지도 제작 당시 장착 일치 여부 → localization fitness 확인 → 반복 복도/평면에서는 quality gate 를 함부로 완화하지 않는다.
제어 PC 가 느림	internal/external/global-map domain 분리 → 제어 domain 의 PointCloud 점검 → elevation 크기/ 해상도/publish rate 조정.
수신 값 해석 오류	매 메시지의 resolution, x_length, y_length 로 shape 를 계산한다. data 는 raw z 가 아니라 0.48-cell_z 계열이다.

운용 완료 기준

외부 domain heartbeat 가 ready 이고, HeightMap 주기가 기대값에 가깝고, data 길이가 width×height 와 같으며, 실제 장애물
위치/방향이 base_link 기준과 일치하면 준비가 완료된 것이다.